

Learning Bault — the study guide

A paced route through [DIVE1](#) to the understanding of someone who built Bault: explain any behaviour, trace any request, debug a failure, extend a subsystem safely, and judge the design. It is built on the verified edition of DIVE1 (19 September 2026, commit [ab67079](#)); every link below goes to a DIVE1 subsection.

How it works

- **42 sessions in four phases.** Phase 0–3 (sessions 1–29) are **Essential** — the foundations and every core flow, about 25 hours. Phase 4 (sessions 30–42) is **Deeper** — the remaining subsystems, operations, testing, history and the defects, about 11 hours. Do them in order: each session assumes the ones before it.
- **Every session has the same parts.**
 - **Objective** — what you can do afterwards.
 - **Read** — the DIVE1 subsections (links).
 - **Open** — the files to keep beside the reading.
 - **Trace** — one exercise done in the code: follow a request or job hop by hop, writing down each `file:line`. You have finished it when you could explain each hop to someone else.
 - **Recall** — five questions, answered from memory, *then* checked against the [answer key](#). Anything wrong: re-read that subsection before moving on.
 - **Checkpoint** — a short task that proves the session stuck.
- **A session is 45–90 minutes.** If one runs long, split it at a Read link; don't skip the Trace.
- **Keep a log.** One line per session: what surprised you, and one question the code raised. The questions become your extension ideas; many will turn out to be in [Appendix D](#).
- **Line numbers drift.** Links and `file:line` refer to commit [ab67079](#). When a line has moved, search for the function named beside it.

Phase 0 — Start here

S1 · Run Bault and meet it · Essential · 60 min

Objective. Have the whole system running locally, know what Bault is and who uses it, and follow one real request from the browser to the database and back.

Read. [§1 What Bault is](#) (all of it) · [§13.1 The local stack](#) · [§13.2 The development loop](#)

Do this first (from the repo root; Docker Desktop running):

```
docker compose -f infra/docker-compose.yml up -d      # Postgres, PgBouncer, MinIO
cp .env.example .env                                  # then set PAYMENT_PROVIDER=sandbox in .env
pnpm install
pnpm --filter @bault/api db:migrate
pnpm db:reset                                          # the clean 9-item Rayquaza seed
pnpm dev                                              # API on :3000, web on :5173
```

Open <http://localhost:5173>, sign in as [red@bault.dev](#) / [11111111](#) (a collector), look at the vault, then sign out and sign in as [hermon@bault.dev](#) (warehouse) and [eldar@bault.dev](#) (admin). Same password for all demo users.

Open. [apps/api/src/main.ts](#) · [apps/api/src/modules/prc/prc.controller.ts](#) · [apps/api/src/modules/prc/price-list.service.ts](#) · [apps/web/src/areas/customer/marketing/LandingPage.tsx](#)

Trace. The landing page's price table. Start at the `api.get('/pricing/list')` call in `LandingPage.tsx`, follow it through the Vite proxy to `prc.controller.ts`, into `PriceListService.publicList`, down to the `pricing_rule` query, and back to how the page picks the trading-card intake rule and prints 5% as "5%" rather than "\$5.00". Then `curl http://localhost:3000/api/v1/pricing/list` and match the JSON to the code.

Recall.

1. What are the three roles, and which console does each use?
2. Name four of the seven invariants in §1.3.
3. Which item states are terminal?
4. Why do the two sites exist, and which one stores goods?
5. What does `pnpm dev` start, and what does it *not* start?

Checkpoint. Without looking, draw the lifecycle diagram from §1.4 and the process diagram from §1.5. Compare with DIVE1.

Phase 1 — Foundations · Essential

S2 · Processes and configuration · 60 min

Objective. Explain how the processes talk and how configuration is loaded and policed.

Read. §2.1 · §2.2 · §2.3 · §2.4

Open. `packages/config/src/env.ts` · `infra/docker-compose.yml` · `apps/api/src/main.ts`

Trace. A boot with `NODE_ENV=production`, `EMAIL_PROVIDER` unset and `PAYMENT_PROVIDER=paypal` with no webhook id: list every validation error `loadEnv` would raise, and the line that raises each.

Recall.

1. Why are there two database URLs?
2. What does `EXPOSE_API_DOCS=false` do, and what did it do before 19 September?
3. Where does `.env` come from when a process runs from `apps/api`?
4. Which settings does the schema refuse in production?
5. What reads `SESSION_COOKIE_SECRET`?

Checkpoint. Explain why `EXPOSE_API_DOCS` is parsed with `booleanFromEnv` and not `z.coerce.boolean()` — what the latter did with the string "false" — and find the test that pins it.

S3 · Boot and the request pipeline · 60 min

Objective. Follow any HTTP request through every layer the API puts it through.

Read. §3.1 · §3.2

Open. `apps/api/src/main.ts` · `apps/api/src/app.module.ts` · `apps/api/src/modules/pay/pay.module.ts`

Trace. A `POST` from `requestContext` to `AllExceptionsFilter`, naming each hop's line: request id, security headers, body limit, throttler, session guard, roles guard, validation pipe, handler, audit interceptor, error filter.

Recall.

1. In what order do the guards run?
2. Why do most modules have no `imports`?
3. Where is `BILLING_PORT` bound, and how?
4. Why does an anonymous `GET /api/v1` get 401?
5. What turns the API docs on?

Checkpoint. Add (in your head, or on a scratch branch you throw away) a new route to an existing controller. List every guard and interceptor it will pass through, in order.

S4 · Transactions and append-only history · 60 min

Objective. Prove which writes share a transaction, and explain exactly what makes history impossible to rewrite — and what does not.

Read. §3.3 · §3.4

Open. `apps/api/src/db/client.ts` · `apps/api/src/db/sql/0001_append_only.sql` · `apps/api/src/modules/cst/custody.service.ts` · `apps/api/src/modules/mkt/purchase.service.ts`

Trace. List every row an intake writes and show that they all share one `tx`.

Recall.

1. What isolation level does the API run at, and what does it add for contested rows?
2. Which SQL operation bypasses the append-only triggers?
3. What does an attempted `UPDATE ledger_record` return to the client?
4. Can the `bault_app` role log in?
5. Why is there an implicit `text → uuid` cast?

Checkpoint. Name the eleven append-only tables from memory, then check against §3.4.

S5 · Migrations and the seed · 45 min

Objective. Add a migration safely and reset the database to the clean state without surprises.

Read. §3.5 · §3.6 · §13.3

Open. `apps/api/src/db/migrate.ts` · `apps/api/src/db/migrations/meta/_journal.json` · `apps/api/src/db/seed.ts`

Trace. Recompute Red's and Golden's seeded balances from the seed's ledger rows.

Recall.

1. What happens to a migration whose journal `when` is lower than the last applied one?
2. Why can't a migration use `CREATE INDEX CONCURRENTLY`?
3. Which table does the seed fail to truncate?
4. Which photographed card is never seeded, and why?
5. Why can the seed truncate append-only tables?

Checkpoint. Write the exact steps to add a column to a table: the SQL file, the journal entry, the Drizzle schema change, and the command that applies it.

S6 · Errors, idempotency and confirmation · 60 min

Objective. Predict the HTTP answer for any failure, and say which operations are protected against being done twice — and how well.

Read. §3.7 · §3.8 · §3.9

Open. `apps/api/src/shared/errors/all-exceptions.filter.ts` · the idempotency and confirmation services under `apps/api/src/shared/`

Trace. Two concurrent purchases with the same idempotency key — once through the marketplace, once through the house store. Where does each end up?

Recall.

1. What does Postgres error `22P02` map to?
2. How does a missing required field read in the error envelope?
3. Is an idempotency key's `expires_at` enforced?
4. What is a confirmation token's lifetime?
5. Which idempotency pattern in the code is race-proof?

Checkpoint. For a new "request a refund" endpoint, decide: idempotency key, confirmation token, or neither — and justify it in two sentences.

S7 · Money, identifiers and observability · 45 min

Objective. Handle money, serials and request ids the way the rest of the code does.

Read. §3.10 · §3.11 · §3.12 · §3.13

Open. `apps/api/src/shared/money.ts` · `apps/api/src/shared/ids.ts` · `apps/api/src/shared/billing/billing.port.ts`

Trace. Compute `applyBasisPoints(99, 500)`, then the chance of an item-serial collision within one millisecond.

Recall.

1. Where does a ledger row's sign live?
2. Which `Money` helpers are unused?
3. What does `feeActionType` do on a billable action?
4. Where is the request id stored while a request runs?
5. Is Sentry wired?

Checkpoint. Explain why amounts are integers in minor units, and what `applyBasisPoints` does with a remainder.

S8 · Accounts, registration and sign-in · 75 min

Objective. Explain an account's life from sign-up to sign-in, and every row a sign-in writes.

Read. §4.1 · §4.2 · §4.3 · §4.4 · §4.5

Open. `apps/api/src/modules/acc/acc.schema.ts` · `auth.controller.ts` · `auth.service.ts` · `verification.service.ts` · `session.service.ts` · `apps/api/src/shared/tokens.ts`

Trace. List every row written by a successful sign-in and by a failed one.

Recall.

1. What stops a username being reassigned?
2. How long does a session last, and is it extended?
3. What is stored for a session token?
4. Which sign-in refusal has its own error code?
5. Why does `login` set `req.user`?

Checkpoint. Sign in through the UI with a wrong password, then find your attempt in `login_attempt` with `psql` (or the admin **Sign-ins** tab as `eldar`).

S9 · Guards, RBAC, suspension, audit and the perimeter · 75 min

Objective. Decide what any user can reach, how it is recorded, and how the edge protects it.

Read. §4.6 · §4.7 · §4.8 · §4.9 · §4.10 · §4.11 · §4.12 · §4.14

Open. `apps/api/src/modules/acc/session-auth.guard.ts` · `apps/api/src/modules/sec/roles.guard.ts` · `allow-suspended.decorator.ts` · `audit.interceptor.ts` · `password.service.ts`

Trace. A suspended user pressing "Sign out" — and why it fails.

Recall.

1. Does a handler-level `@Roles` combine with a class-level one?
2. Which failed requests are audited?
3. Which sessions does a password reset keep? A password change?
4. What is the throttler's bucket key, and which bucket caps every route?
5. Why is `TRUST_PROXY=true` refused?

Checkpoint. With the default config, what limits apply per IP to `POST /auth/login` and to `GET /pricing/list`? How does the `auth` throttler know to skip the second?

Phase 1 mastery check. Without notes, explain to an imaginary new engineer: what happens between a browser click and a database write; which history can never change and why; and what a suspended user can still do. Then check each claim against §3 and §4.

Phase 2 — The life of a card · Essential

S10 · The custody model and kernel · 60 min

Objective. Explain how an item's owner, shelf and state change, and what is guaranteed about it.

Read. §5.1 · §5.3 · §5.4 · §5.5

Open. `apps/api/src/modules/cst/cst.schema.ts` · `lifecycle.ts` · `custody.service.ts`

Trace. A marketplace purchase's custody writes, through `transferOwnership` and `changeState`.

Recall.

1. What stops an item row being deleted?
2. Is `stored → stored` legal?
3. Which custody event type is never written?
4. Is `item_change_history` append-only?
5. Where is the hold flag enforced?

Checkpoint. Find the one place that writes owner, shelf and state without going through the kernel.

S11 · Facilities, bins and parcels · 60 min

Objective. Follow a physical box from arrival to opening.

Read. §5.6 · §5.7 · §5.10

Open. `apps/api/src/modules/inv/parcel.service.ts` · `facility.service.ts` · `apps/api/src/modules/cst/stow.service.ts`

Trace. A box addressed to "nosuchperson": receive, claim, open, process.

Recall.

1. When is the parcel processing fee charged?
2. What happens to an empty parcel at process?
3. Can an unclaimed parcel be opened?
4. How is the auto-stow shelf chosen?
5. Are placeholder facility addresses shown?

Checkpoint. As `hermon`, receive a parcel on the warehouse console and find its `parcel_event` rows.

S12 · Intake and lots · 60 min

Objective. Book any kind of item in, and say exactly what it costs and what it writes.

Read. §5.2 · §5.8 · §5.9 · §5.11

Open. `apps/api/src/modules/inv/intake.service.ts` · `item-classes.ts` · `apps/api/src/modules/pay/billing.service.ts` · `apps/api/src/modules/mem/tiers.ts`

Trace. Book a lot of 3 cards and a lot of 40; list every row written for each.

Recall.

1. What does a card lot of 5 become?
2. What is `intake_lot`'s fallback price?
3. Does a covered intake write a charge?
4. Is a box of units all-or-nothing?
5. What does a broken lot keep doing?

Checkpoint. Book a single card as `hermon` for `red` and find its charge (\$1) in the ledger.

S13 · Pricing rules and the price list · 45 min

Objective. Resolve any price the way the billing engine does.

Read. §6.1 · §6.2 · §6.3

Open. `apps/api/src/modules/prc/pricing.service.ts` · `price-list.service.ts` · the pricing rules in `apps/api/src/db/seed.ts`

Trace. Resolve `price('intake', { itemClass: 'graded_slab' })` against the seed; then imagine a newer catch-all at \$3 and resolve again.

Recall.

1. Which wins: the class-specific rule or the newest rule?
2. What does a percentage rule with no base cost?
3. How is a price withdrawn?
4. Where does an unknown action type land on the price list?
5. What does `tryPrice` swallow?

Checkpoint. Explain why the landing page shows \$1.00 for "Booking one card in" when the catch-all intake rule is \$5.

S14 · The billing chokepoint · 60 min

Objective. Explain every fixed-price charge in the system, including when it is free.

Read. §6.4 (with its worked example) · §3.10

Open. `apps/api/src/modules/pay/billing.service.ts` · `apps/api/src/modules/mem/tiers.ts` · `apps/api/src/modules/mem/membership.service.ts`

Trace. A Registry member's 11th lot intake, end to end.

Recall.

1. Why is the allowance checked before the price?
2. Which allowance does a lot consume?
3. What does a zero price write?
4. Which ledger type does `BillingService` write?
5. Does `BillingService` check the balance?

Checkpoint. List four fees that do *not* go through `BillingService.charge`, and who charges each.

S15 · The ledger and debt · 60 min

Objective. Derive any balance, and explain what happens to an account that goes negative.

Read. §6.5 · §6.6

Open. `apps/api/src/modules/pay/ledger.service.ts` · `apps/worker/src/jobs/debt.ts` · `interest-accrual.ts` · `wallet-suspension.ts` · `ledger-invariant-check.ts`

Trace. Run the `negativeAccounts` logic by hand on a ledger that goes negative, recovers, then goes negative again.

Recall.

1. Which negative crossing starts the interest clock?
2. What is the minimum daily interest?
3. Is an account at exactly -\$20.00 suspended?
4. When is an account reinstated?
5. What does the invariant job flag?

Checkpoint. Explain why a re-run of the interest job on the same day is a defect (D.1).

S16 · Storage and the vault · 60 min

Objective. Predict every storage charge, and what a collector sees in their vault.

Read. §5.12 · §5.13 · §5.14 · §5.15

Open. `apps/worker/src/jobs/storage-fee.ts` · `apps/api/src/modules/vlt/vault.service.ts` · `storage-policy.ts` · `break-even.service.ts`

Trace. Compute the storage charges for a sealed case covered by a membership that lapses on day 608.

Recall.

1. On which day is a card first billed for storage?
2. What is the storage base for an item with no intake charge?
3. Are listed items billed?
4. Where does a shipped item appear in the vault?
5. What keeps the storage sweep from billing twice?

Checkpoint. Explain how `storage_period_cover` keeps a period a membership covered from being billed after the membership ends, and find the two statements in the sweep that write and read it.

S17 · Listings, browsing and the atomic sale · 75 min

Objective. Follow a sale from listing to the ledger rows, and name every guarantee and gap.

Read. §7.1 · §7.2 · §7.3 · §7.4 · §7.11

Open. `apps/api/src/modules/mkt/listing.service.ts` · `browse.service.ts` · `purchase.service.ts` · `tests/concurrency/no-double-sale.test.ts`

Trace. `POST /marketplace/listings` down to the custody event; then a \$160-fee sale by a Registry seller who has already sold \$950 this cycle — every ledger row.

Recall.

1. What stops a card being listed twice?
2. Does opening a dispute freeze an item?
3. Where is the lock that prevents a double sale?
4. Why is the seller credited gross and then debited the fee?
5. Which race remains open on a purchase?

Checkpoint. As `red`, list a card; as `golden`, buy it. Find the transaction, both ledger movements and the custody event.

S18 · Offers, swaps and gifts · 60 min

Objective. Explain negotiation and exchange between collectors, and where they go wrong.

Read. §7.5 · §7.6

Open. `apps/api/src/modules/mkt/offer.service.ts` · `trade.service.ts` · `tests3/integration/band2-negotiation.test.ts`

Trace. A counter-offer from `respond` to the notification recipient — and why the wrong person is notified.

Recall.

1. Who may accept an offer?
2. Why does the purchase run before the offer's status is written?
3. Do offers expire?
4. What does approving a swap fail to re-check?
5. How is a gift modelled?

Checkpoint. Write the test you would add to catch the swap re-check defect (as prose; don't commit it).

S19 · The parcel model: weight, boxes, carriers · 60 min

Objective. Size and weigh any outbound parcel the way the quote does.

Read. §9.1 · §9.2 · §9.4 · §9.5

Open. `apps/api/src/modules/shp/boxes.ts` · `carriers.ts` · `parcel-profile.service.ts` · `packages/adapters/src/shipping.ts`

Trace. A 500 g sealed box going to Japan through `chooseBox` → `dimensionalGrams` → `billableGrams` → `checkService`, including the ePacket dimension check.

Recall.

1. Why 167 and not 139?
2. What comes first: rounding or comparing actual and dimensional weight?
3. Where does a parcel's size come from?
4. When is `direct_overnight` eligible in a normal quote?
5. Which boxes break ePacket's limits even when empty?

Checkpoint. Compute the billable weight of a 60 g slab in a small box, by hand.

S20 · Quote, choose, settle · 75 min

Objective. Produce a quote by hand, and explain how it is paid for.

Read. §9.6 · §9.7 · §9.8 · §9.9 · §9.10

Open. `apps/api/src/modules/shp/shipment.service.ts` · `shipping-options.ts` · `apps/api/src/modules/mem/membership.service.ts`

Trace. Reproduce the worked example in §9.9, then redo it for a Registry member.

Recall.

1. What is the insurance premium?
2. When is a signature forced?
3. What is the `pickBest` score?
4. When is membership cover spent?
5. What happens when the wallet is short?

Checkpoint. As `red`, quote one card to a US address in the web app and match every number to your hand calculation.

S21 · Destinations, dispatch and after · 60 min

Objective. Follow a paid shipment until the label is bought and tracking runs.

Read. §9.3 · §9.11 · §9.12 · §9.18 · §9.19

Open. `apps/api/src/modules/shp/dispatch.service.ts` · `labelRequest` in `shipment.service.ts` · `packages/adapters/src/easypost.ts` · `apps/worker/src/jobs/tracking-refresh.ts`

Trace. `POST dispatch` from the row lock to the outbox event. Name the side effect that can escape a rollback.

Recall.

1. What must the scan set satisfy?
2. What weight goes on the label?
3. Why can't EasyPost re-rate at label time?
4. Why do parcels never reach `delivered`?
5. What locks the payment-expiry sweep?

Checkpoint. Explain how the dispatch screen builds `scannedItemIds` from barcode scans, and what happens — on the screen and at the API — when a card that isn't in the shipment is scanned.

Phase 2 mastery check. Trace one card's whole life in writing: arrives in a parcel, is booked in, is stored past its included period, is sold, and is shipped to the buyer — every service call, every row, every fee, with `file:line`. Check it against §5, §6, §7 and §9.

Phase 3 — The system around it · Essential

S22 · Wallet requests · 60 min

Objective. Run a cash-in and a cash-out through review, and name every row they write.

Read. §6.7 · §6.12

Open. `apps/api/src/modules/pay/wallet-request.rules.ts` · `wallet-request.service.ts` · `wallet-request.controller.ts`

Trace. `complete` for a cash-in: every row it writes.

Recall.

1. Can the same admin approve and complete a request?
2. What does submit write?
3. What prevents double settlement?
4. What does a non-owner get when opening someone's request?
5. What makes two requests duplicates?

Checkpoint. As `golden`, request a cash-in; as `eldar`, approve and complete it. Find the events and the ledger row.

S23 · Membership tiers and allowances · 60 min

Objective. Say for any action whether a member pays, and how the allowance is spent.

Read. §10.1 · §10.2 · §10.5

Open. `apps/api/src/modules/mem/tiers.ts` · `mem.schema.ts` · `membership.service.ts` · `apps/api/src/modules/pay/billing.service.ts`

Trace. An intake by a Folio member through `BillingService.charge`, `consume`, the conditional `jsonb_set` and the early return.

Recall.

1. Why is `consume` conditional and checked?
2. What does `ALLOWANCE_ALIASES` do?
3. What does `UNLIMITED` equal?
4. Why does `remaining()` return 0 for an action the tier doesn't name?
5. Who waives the marketplace commission?

Checkpoint. For a Registry member, list what their 11th intake, 3rd inspection and first \$1,200 sale cost.

S24 · Subscribe, cancel, renew · 60 min

Objective. Explain every membership change and its money.

Read. §10.3 · §10.4 · §10.6 · §10.7

Open. `apps/api/src/modules/mem/membership.service.ts` · `apps/worker/src/jobs/membership-renewal.ts` · migration `0030_a_downgrade_is_not_a_cancellation.sql`

Trace. An upgrade from Registry to Trust on day 10 of a cycle whose fee was 17,560.

Recall.

1. How does a downgrade take effect?
2. What does "keep" cost?
3. Why `date_trunc('milliseconds', ...)`?
4. What does "lapsed" mean?
5. What happens at renewal when there is no pricing rule?

Checkpoint. Explain the renewal-vs-storage-sweep timing defect (D.1).

S25 · The outbox and notifications · 60 min

Objective. Follow any event from the transaction that emits it to the collector's inbox and email.

Read. §10.8 · §10.9 · §10.10 · §10.11 · §10.12 · §10.13

Open. the outbox service under `apps/api/src/modules/not/outbox/` · `event-types.ts` · `apps/worker/src/jobs/outbox-dispatch.ts` · `notification-message.ts`

Trace. A staff support reply from `outbox.emit` to an in-app row plus an email row.

Recall.

1. Which two events cannot be switched off in-app?
2. What is the default for an event not in the catalogue?
3. Who gets email?
4. What is the delivery guarantee?
5. Where does the FAQ live?

Checkpoint. Start `pnpm dev:worker`, trigger an event (e.g. list a card), and watch it arrive in the notifications page.

S26 · The background worker · 45 min

Objective. Know every job, when it runs, and which ones are unsafe to re-run.

Read. §11 (all)

Open. `apps/worker/src/index.ts` · `apps/worker/src/jobs/registry.ts` · every file in `apps/worker/src/jobs/`

Trace. For each job, what a pg-boss retry after a mid-run failure would do.

Recall.

1. What timezone are the schedules in?
2. How is a double tick prevented across processes?
3. What are pg-boss's retry defaults here?
4. Why does suspension run at 03:15?
5. Which jobs are unsafe to re-run?

Checkpoint. Draw the nightly timeline (01:00–04:00 UTC) with every job on it.

S27 · The web app: serving, boot, routing and data · 75 min

Objective. Explain how the SPA starts, decides what to show, and talks to the API.

Read. §12.1 · §12.2 · §12.3 · §12.4 · §12.5 · §12.6

Open. `apps/web/vite.config.ts` · `apps/web/proxy-target.ts` · `apps/web/src/main.tsx` · `App.tsx` · `shared/session.ts` · `shared/routing.ts` · `shared/api.ts`

Trace. A collector who opens `#/services` with an expired cookie.

Recall.

1. Why does `vite.config.ts` delete `VITE_USER_NODE_ENV`?
2. What does the proxy answer when the API is down?
3. What is in the preview-gate cookie?
4. What does a 401 at boot give? A 503?
5. What does a suspended account see?

Checkpoint. Stop the API while the web app is open, and explain what you see.

S28 · Two languages and the design system · 60 min

Objective. Add a string and a styled element the way the project requires.

Read. §12.7 · §12.8 · §12.9

Open. `apps/web/src/shared/i18n.tsx` · `apps/web/src/shared/theme.tsx` · `apps/web/src/index.css` · `DESIGN.md` · `scripts/design-lint.mjs`

Trace. Add a key to the Hebrew catalogue only, and predict the failure.

Recall.

1. What is the default locale on a first visit?
2. How does the theme mark "system"?
3. What are the three densities, and what do they change?

4. Is money set in mono?
5. Does CI run design-lint?

Checkpoint. Run `node scripts/design-lint.mjs`, then (on a scratch copy) add `color: #ff0000` to a rule and see it fail.

S29 · The signed-out surface and the areas · 75 min

Objective. Find the code behind any screen and trace a screen's full flow.

Read. §12.10 · §12.11 · §12.12 · §12.13 · §12.14 · §12.15

Open. `LandingPage.tsx` · `DemoSlab.tsx` · `apps/web/src/areas/customer/shipping/ShipmentComposer.tsx` · `VaultPage.tsx` · both consoles

Trace. The shipment composer when create succeeds and select-rate fails — and why a retry answers 409.

Recall.

1. Which intake price is the landing page's headline?
2. What happens if `/membership/tiers` fails?
3. What is real on the demo slab?
4. How are the demo users removed from production builds?
5. Which purchases send an `Idempotency-Key`?

Checkpoint. Pick any screen at random and, within five minutes, name the file that renders it and the API route it calls.

Phase 3 mastery check (the essential path is done). Answer, with `file:line` evidence: (1) Why can a Trust member ship a \$2,000 card with no insurance charge but pay for a \$3,000 one? (2) What stops a collector with a –\$50 balance from starting a new service, and when does that stop? (3) What would break if you deleted `apps/api/src/db/sql/0001_append_only.sql`?

Phase 4 — Deeper · Optional

S30 · Item services: framework, fees and fulfilment · 60 min

Read. §8.1–§8.6 · **Open.** `apps/api/src/modules/dis/dis.schema.ts` · `service.service.ts` · `dis.controller.ts`

Trace. `POST /services/photography` to the INSERT; then a second call and where its 409 comes from. **Recall.** 1. The four statuses and what each means to an operator? 2. The order of `create`'s steps? 3. Which requests skip the duplicate guard? 4. Is accept race-safe? Is complete? 5. When is a membership allowance spent for a service, and is it returned on deny? **Checkpoint.** Work out what a Trust member pays for their 9th inspection and for a PSA Express submission.

S31 · Grading · 60 min

Read. §8.9 · §8.10 · §8.11 · **Open.** `grading-tiers.ts` · `grading.service.ts` · `grading-submission.schema.ts` **Trace.** Follow `tests/integration/dis-item-services.test.ts:107-169` hop by hop. **Recall.** 1. Is \$4,999.00 allowed on Express and on Walkthrough? 2. What happens when an admin refuses approval? 3. When does a card become `at_grader`? 4. When can a submission close? 5. Can grading complete without shipping? **Checkpoint.** Name the two ways a card can get stuck because of a grading batch.

S32 · Media, inspection, items that leave, custom requests · 75 min

Read. §8.7 · §8.8 · §8.12–§8.17 · **Open.** `media.service.ts` · `disposal-services.service.ts` · `donation.service.ts` · `lot-split.service.ts` · `custom-request.service.ts` **Trace.** A donation confirm — every row it writes. **Recall.** 1. Which table holds videos? 2. Which items does de-slab refuse, and what does it do to the grade? 3. What is the cull's window and cost? 4. Who owns a donated item? 5. When is a custom request billed, and why not through the billing port? **Checkpoint.** Show how two concurrent accept-quote calls could debit twice.

S33 · House store, consignment and buyout · 60 min

Read. §7.7 · §7.8 · §7.9 · **Open.** `house-store.service.ts` · `consignment.service.ts` · `consignment-channels.ts` · `buyout.service.ts` **Trace.** A house-store purchase and its stow — every custody event. **Recall.** 1. When is a store item created? 2. Why is the store's idempotency key random? 3. The consignment channel cuts? 4. What moves ownership in a buyout? 5. What is wrong with an open consignment? **Checkpoint.** Explain why "the platform account" owns consigned and bought-out items.

S34 · Escrow · 60 min

Read. §7.10 · §7.12 · **Open.** `escrow-terms.ts` · `escrow.service.ts` · `tests/integration/esc-and-human-fulfilment.test.ts` **Trace.** A \$9,000 deal raised by a Trust member, from raise to settle — every ledger row and event. **Recall.** 1. The floor and the fee? 2. What does "held" mean in the ledger? 3. How is an external party's release recorded? 4. The fee after the Trust waiver? 5. Why is escrow unsafe under concurrency? **Checkpoint.** List the four escrow defects in D.1/D.3 and the lines behind them.

S35 · Provider money: cash-out, top-ups, chargebacks · 60 min

Read. §6.8–§6.11 · §6.13 · §2.5 · **Open.** `money-terms.ts` · `checkout.service.ts` · `chargeback.service.ts` · `packages/adapters/src/payment.ts` **Trace.** A \$200 cash-out end to end — and show that the wallet loses exactly \$200: a \$193 withdrawal row and a \$7 fee row. **Recall.** 1. The fee on \$100.01? 2. What happens when the settled amount doesn't match the request? 3. What gives single-credit idempotency for a top-up? 4. Which ledger type does a chargeback write? 5. What does the webhook do today? **Checkpoint.** Explain what the code did before 19 September (it took the fee twice) and which test assertion now prevents it.

S36 · Shipping changes, shared parcels and other routes out · 75 min

Read. §9.13–§9.17 · §9.20 · **Open.** `shipment-edit.service.ts` · `group-shipment.service.ts` · `customs.service.ts` · `direct-ship.service.ts` · `human-fulfilment.service.ts` **Trace.** Merge two requests with different boxes and insured values; compute the result. Then apportion \$100 declared across a 60 g slab and two 5 g cards. **Recall.** 1. When can a shipment be edited? 2. When is the \$25 restocking fee charged? 3. How does merge combine insurance? 4. Does the group payer pay for everyone? 5. What closes a hand delivery? **Checkpoint.** Explain why a direct-ship shipment can never be dispatched.

S37 · Support and administration · 45 min

Read. §10.14 · §10.15 · §10.16 · **Open.** `support.service.ts` · `sup.controller.ts` · `adm.service.ts` · `shelf-yield.service.ts` **Trace.** A debt-suspended user opening a ticket, through the guard and `@AllowSuspended`. **Recall.** 1. What does a ticket's status mean? 2. Why `notFound` rather than `forbidden` for someone else's ticket? 3. What stops an admin locking themselves out? 4. What does shelf yield measure? 5. Where does commission revenue for shelf yield come from? **Checkpoint.** As `eldar`, open the Shelf yield tab and explain one number on it from the code.

S38 · The provider adapters · 60 min

Read. §2.5–§2.11 · **Open.** `packages/adapters/src/payment.ts` · `shipping.ts` · `easypost.ts` · `storage/s3.ts` (under `packages/adapters/src/`) · `apps/api/src/shared/adapters/adapters.module.ts` **Trace.** A \$1 PayPal approval submitted as a \$5,000 top-up, through `checkout.service.ts` — where is it caught? **Recall.** 1. Why is a top-up a capture? 2. What does `PayPal-Request-Id` carry? 3. How does EasyPost authenticate? 4. What does the worker's tracking job use? 5. How many times is the payment sandbox refused in production? **Checkpoint.** List what would have to change for EasyPost tracking to work end to end.

S39 · Operations: tunnel, images, CI, backups, readiness · 75 min

Read. §13.4–§13.12 · **Open.** `scripts/tunnel.mjs` · the three Dockerfiles · `apps/web/nginx.conf` · `.github/workflows/ci.yml` · `infra/ops/backup.sh` · `docs/production-readiness.md` **Trace.** `curl -I` the web image's `/` and a font file (reason it through the nginx config), and explain why the headers differ. **Recall.** 1. Why tunnel `vite preview` and not the dev server? 2. What does the tunnel wait for before opening? 3. Why is MinIO a CI step and not a service? 4. What must `TRUST_PROXY` be behind the nginx container? 5. Why did the API image's migrator fail, and

how does it find the SQL now? **Checkpoint.** Rank the top three production blockers from §13.9 and Appendix D, with one sentence each.

S40 • Testing • 60 min

Read. §14 (all) • **Open.** `vitest.workspace.ts` • `scripts/test.mjs` • `tests/integration/helpers/http.ts` **Trace.** `pnpm test` from its suite list to the reseed, and the exit status when `core` fails. Then follow `fundWallet(red, 100000)` through its three HTTP calls to the one ledger row. **Recall.** 1. Why reseed even on failure? 2. Which projects need a live API? 3. Why can't `fundWallet` fund the admin? 4. Why is the credentials-in-bundle test vacuous in CI? 5. What is the root cause of the known flakiness? **Checkpoint.** Run one integration test file alone (§14.9), then `pnpm db:reset`.

S41 • The design history • 60 min

Read. Appendix B (all) • **Open.** whatever each decision points to **Trace.** One intake from `intake.service.ts` through the custody lock, `billing.charge` and `outbox.emit`, naming the historical decision behind each step. **Recall.** 1. Why is there no balance column? 2. Why is there no bin capacity? 3. Who may accept an offer, and why that rule? 4. Why a random idempotency key in the house store? 5. Why does `vite.config.ts` delete `VITE_USER_NODE_ENV`? **Checkpoint.** Pick three decisions in Appendix B and argue the opposite choice for each — what would it have cost?

S42 • Capstone: the defects • 90 min

Read. Appendix D (all) and Appendix C **Trace.** For five defects of your choice — at least one each from D.1, D.2 and D.3 — find the code, reproduce the reasoning, and write the fix you would make and the test that would prove it. **Checkpoint (final mastery).** You are done when you can, without notes: (1) explain any screen's behaviour from the database up; (2) predict the exact money movement of any action for a member and a non-member; (3) name which invariants the code keeps, which it only intends, and where it breaks them; (4) propose a change to any subsystem and list every file, table, test and doc section it touches.

Answer key

S1. 1. `user` (collector areas), `warehouse_operator` (warehouse console), `admin` (admin console). 2. Any four of: items never deleted; one owner; append-only history; balance = sum of ledger; prices frozen onto charges; one chokepoint for fixed-price actions; successful state changes audited. 3. `shipped`, `donated`, `consigned`, `discarded`. 4. Delaware levies no sales tax, so it is a forwarding site; New Jersey stores goods. 5. The API and the web app; not the worker (`pnpm dev:worker`).

S2. 1. PgBouncer's transaction pooling can't carry pg-boss LISTEN/NOTIFY or DDL, so jobs and migrations use a direct URL. 2. Nothing — it is off. It is parsed strictly by `booleanFromEnv`; it used to *enable* the docs, because `z.coerce.boolean` reads "false" as true. 3. `loadDotenvFromRoot` walks up to six directories to the root `.env`. 4. Sandbox storage, shipping or payment; console email; docs exposed without a password. 5. Nothing.

S3. 1. Throttler, then session, then roles. 2. Cross-module services come from `@Global` kernels. 3. `pay.module.ts:31`, with `useExisting`. 4. It isn't `@Public`. 5. `EXPOSE_API_DOCS` set to `true`, `1`, `yes` or `on` (plus Basic auth when a password is set).

S4. 1. READ COMMITTED, plus `SELECT ... FOR UPDATE` on contested rows. 2. `TRUNCATE`. 3. A 500 `internal` (the trigger raises `23514`). 4. No — it has NOLOGIN, and the app connects as the superuser `baul`. 5. Reference columns are `text` while keys are `uuid`, so joins need the cast.

S5. 1. It is skipped silently. 2. Pending migrations run in one transaction. 3. `parcel_photo`. 4. `SN-EVS218-0010`, kept for the intake bench. 5. The guards are row triggers, which don't fire on `TRUNCATE`.

S6. 1. 400 `validation_failed`. 2. "X is required." 3. No. 4. 300 seconds. 5. The unique provider reference (`providerRef` with a unique index).

S7. 1. In `direction` (debit/credit); amounts are positive. 2. `add`, `subtract`, `sum`, `zero`. 3. A narrower price rule is tried first, falling back to the action's own. 4. In `AsyncLocalStorage`. 5. No — `SENTRY_DSN` is read by nothing.

S8. 1. A unique index, a CHECK constraint, and the `user_account_username_immutable` trigger. 2. 7 days, absolute, never extended. 3. Only its SHA-256. 4. `email_unverified`. 5. So the audit row names the person signing in.

S9. 1. No — it overrides it. 2. None. 3. A reset keeps none; a change keeps the caller's own. 4. Controller, handler, throttler name and `req.ip`; the `auth` bucket (30/min by default) applies only to routes marked `@AuthBucket` — before 19 September it capped every route. 5. It believes a forged `X-Forwarded-For`.

S10. 1. The `trg_no_delete_item` trigger. 2. Yes — a no-op. 3. `dispatch`. 4. No. 5. In each caller, plus `relocate`.

S11. 1. At `process`, once. 2. A 409 unless the operator gives a reason. 3. No. 4. The shelf with the fewest items, ties broken by barcode. 5. Hidden in production only.

S12. 1. Five separate items at \$1 each. 2. The item class's own intake rule. 3. No. 4. Only through the validate-first pass. 5. It keeps being billed for storage.

S13. 1. The class-specific rule; "newest" only breaks ties within a class. 2. \$0. 3. Insert a new rule with value 0 — `effective_to` is never set. 4. `services`. 5. Every error.

S14. 1. An included but unpriced action would otherwise throw. 2. `intake`, through `ALLOWANCE_ALIASES`. 3. Nothing. 4. `service_charge` (or `fee` for `marketplace_fee`). 5. No.

S15. 1. The last one. 2. 1 cent. 3. No — the test is strictly below $-\$20$. 4. When the balance is $\geq -\$20$ and the account carries the auto-suspension marker. 5. Amounts ≤ 0 , and settled charges with no ledger row.

S16. 1. Day 180. 2. The storage rule's flat `value`. 3. No. 4. Nowhere. 5. It counts the periods already settled — storage charges plus membership-covered periods — against the periods elapsed.

S17. 1. The item's `listed` state, taken under an item row lock. 2. No — only the hold flag freezes. 3. `FOR UPDATE` on the listing (then the item). 4. So the statement shows gross – fee = net. 5. Concurrent debits against the buyer's unlocked wallet.

S18. 1. Anyone except the party who proposed the current price. 2. A failed purchase leaves the offer `pending`. 3. No. 4. The items' ownership, state and hold. 5. A swap with an empty `requested` list.

S19. 1. It is the reference service's published divisor; 139 over-quotes by about 20%. 2. Take the larger of actual and dimensional weight first, then round up. 3. The predicted box — items are never measured. 4. Never — a normal quote has no origin facility. 5. Large and extra-large (the 36-inch sum).

S20. 1. 1.5% of insured value, minimum \$2. 2. When insured value is above \$500. 3. Total + postage credit + $\text{transitDaysMax} \times \2.50 . 4. At charge time, in the same transaction, through a conditional update. 5. The shipment goes to `awaiting_payment` for 7 days, with nothing charged.

S21. 1. Exact set equality with the shipment's items. 2. The scale reading minus the box's weight (or the estimate). 3. It buys the quoted rate by id. 4. The worker's tracking job uses the sandbox, which always answers `in_transit`. 5. `SELECT ... FOR UPDATE` in one transaction.

S22. 1. Yes, if they are not the requester. 2. The request, one event, one outbox row and one audit row — no ledger. 3. The row lock, the terminal status, and the partial unique index on `settled_ledger_id`. 4. 404. 5. Same type, amount, currency and reference while the first is still open.

S23. 1. It prevents a lost update, and stops a missing period row being answered as "covered". 2. Makes `intake_lot` draw on the `intake` allowance. 3. –1. 4. "Not in this tier" and "used up" both mean it will be charged. 5. `waive('marketplace_fee')` in `purchase.service.ts`, proportionally at the cap.

S24. 1. Through `scheduled_tier`, applied at renewal via `coalesce`. 2. Nothing. 3. The period row is looked up by exact equality through a JS `Date` (milliseconds), while Postgres stores microseconds. 4. The window has passed and it isn't renewed yet — everything bills at the ordinary price. 5. The membership is left lapsed.

S25. 1. `parcel_damaged` and `arrival_not_accepted`. 2. In-app on, email off. 3. Active accounts only. 4. At-least-once. 5. As static modules in the SPA.

S26. 1. UTC. 2. A singleton key per 60 seconds. 3. 2 retries, 0 delay, 15-minute expiry. 4. So it sees the day's interest. 5. Interest accrual always; outbox dispatch and renewal if runs overlap.

S27. 1. The root `.env`'s `NODE_ENV=development` made `vite build` produce a development bundle. 2. 503 with `api_unreachable`. 3. An HMAC under a random key made at startup — never the password. 4. 401 → `anonymous` (the sign-in form); 503 → `blocked` with a Retry button. 5. Only support (and their profile).

S28. 1. English (`en`). 2. It stamps no `data-theme`. 3. vault (default), `marketing` and `warehouse`; they change only the spacing variables. 4. No — it uses `tabular-nums`. 5. No.

S29. 1. The `trading_card` intake rule. 2. The tiers section is omitted. 3. Only the photograph. 4. `DEV` is folded to `false` at build time, so the module is dropped. 5. Only the house store; the top-up sends one in the body.

S30. 1. `requested` = pending, `in_progress` = accepted, `completed` = done, `cancelled` = denied. 2. Blocked-balance check, duplicate guard, bill, insert. 3. Requests with no item id, and custom requests. 4. Accept is race-safe (lock, compare, update); complete is not (it checks a read made before the lock). 5. At creation; it is not returned on deny.

S31. 1. Yes, on both. 2. The request is cancelled, with no refund. 3. When the batch ships. 4. When none of its members is still requested or in progress. 5. Yes — the item's state is left unchanged.

S32. 1. `item_image`, type `video`. 2. Items with no grade or a grade of "raw"; it overwrites the grade with the operator's free text. 3. 30 days from `received_at`; free. 4. `platform@bault.dev`. 5. When the collector accepts the quote; it has no pricing rule, so it writes a `fee` ledger debit directly.

S33. 1. At payment, as `received` with no bin. 2. A product has copies, so a key per product would block the second copy. 3. Card show 10%, auction house 1%, eBay partner 1%, falling back to 5%. 4. The collector's accept. 5. The item isn't reserved.

S34. 1. \$500 floor; 1% with a \$25 minimum. 2. An `escrow_hold` debit. 3. By staff, with `side` naming the external side, in the attested-by column. 4. \$40 — the \$90 gross fee minus \$50 waived on the \$5,000 cap. 5. No row locks, so a deal can be funded or settled twice.

S35. 1. 601 (cents). 2. 409, and nothing is written. 3. `providerRef` plus its unique index. 4. `chargeback`, plus a `fee` row. 5. Verifies the delivery, then nothing.

S36. 1. Only while `requested`. 2. Only when cancelling from `rates_selected`. 3. Summed, capped at \$5,000. 4. No. 5. A hand-over scan, which moves it to `delivered`.

S37. 1. Whose turn it is. 2. Not to confirm the ticket exists. 3. The self-status and self-role guard in `updateUser`. 4. Revenue per item-month — never margin. 5. The `ledger_record` fee rows, joined through the listing.

S38. 1. An approved order id can't be invented, so the server captures what the provider approved. 2. The idempotency key. 3. Basic auth with the key as the username and an empty password. 4. `SandboxShippingAdapter`. 5. Twice — by the schema and by the factory.

S39. 1. The dev server serves source files. 2. A 401 from the preview (the password gate is up). 3. The official image needs `server /data` as its command, which a service container can't pass. 4. `uniqueLocal` or the proxy's CIDR — never `true`. 5. It doesn't any more: it looked only for `dist/db/sql/0001_append_only.sql`, which is never in the image, and now also tries `src/db/sql/` under the working directory.

S40. 1. A failed run leaves more residue, not less. 2. `integration`, `core`, `concurrency`, `property`. 3. Separation of duties — an admin can't complete their own request. 4. CI never builds the web app, so there is no `dist/` to check. 5. No per-test isolation: suites share the seeded database.

S41. 1. A stored balance could drift from the ledger; the sum is the truth. 2. Capacity was never enforced and was the wrong model; the directed stow picks the emptiest shelf instead. 3. Not the party who proposed the price — so nobody can accept their own number. 4. Products have copies. 5. The root `.env` turned every build into a development build.